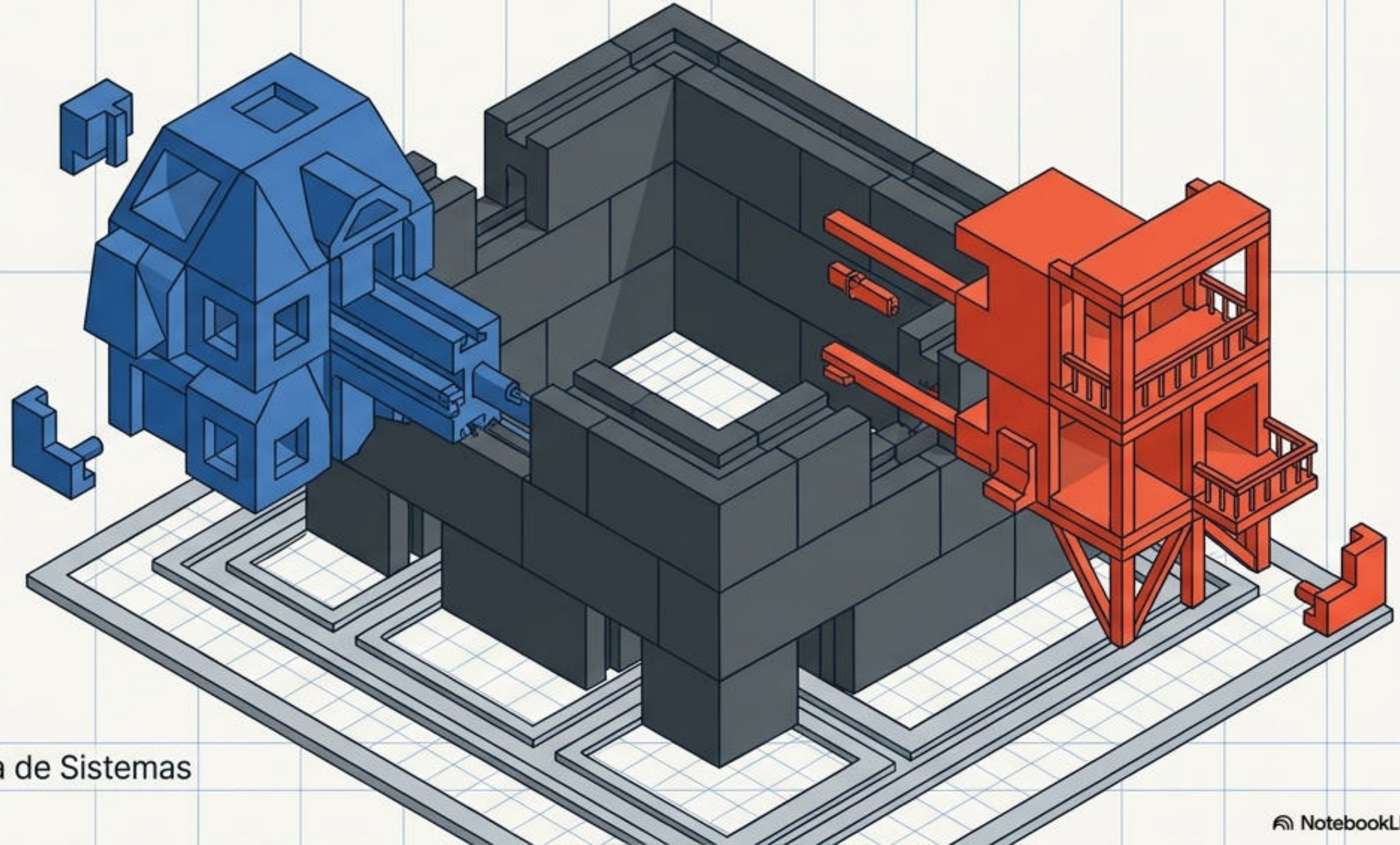
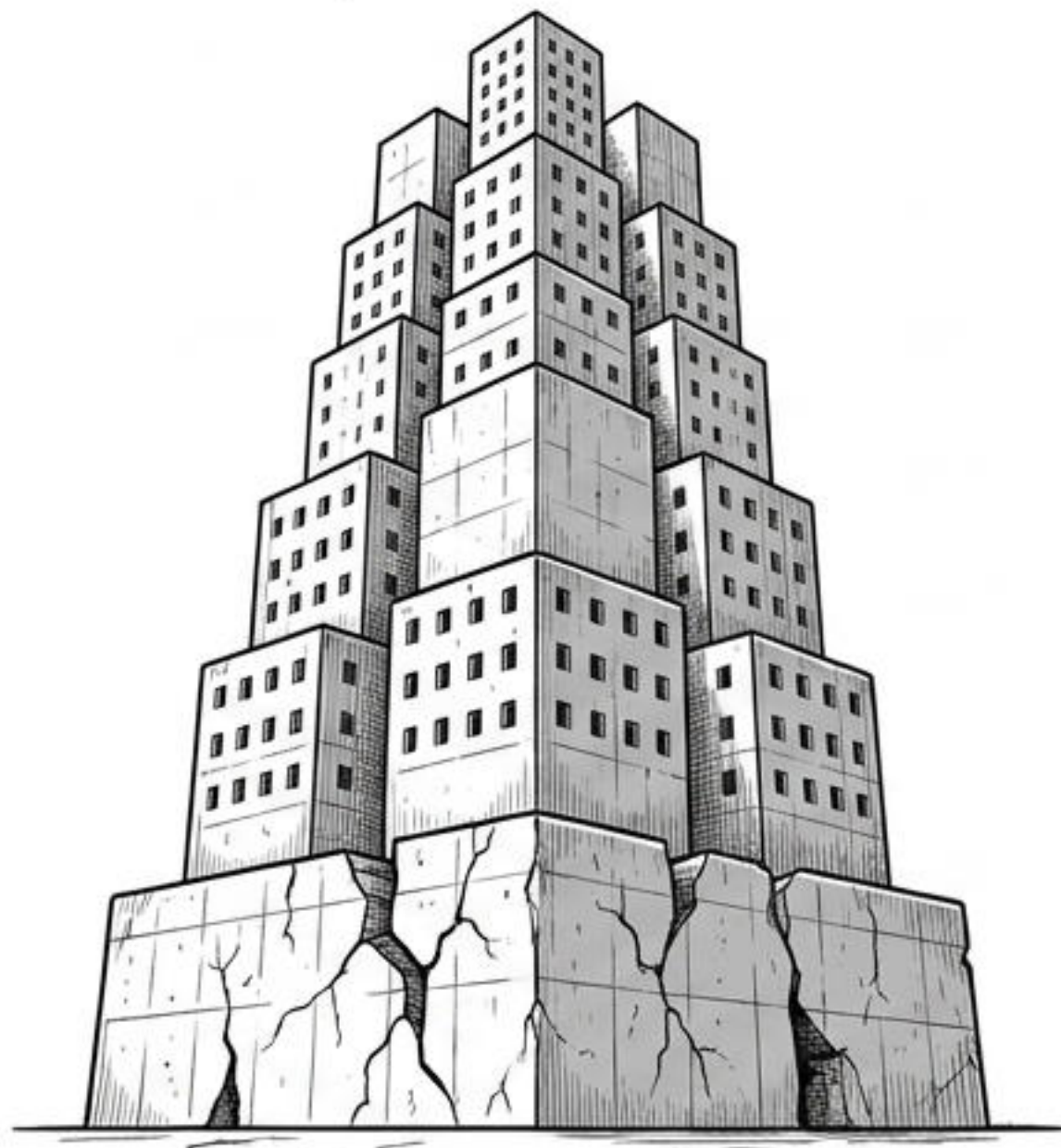


Pilares de la POO: Interfaz y Delegación

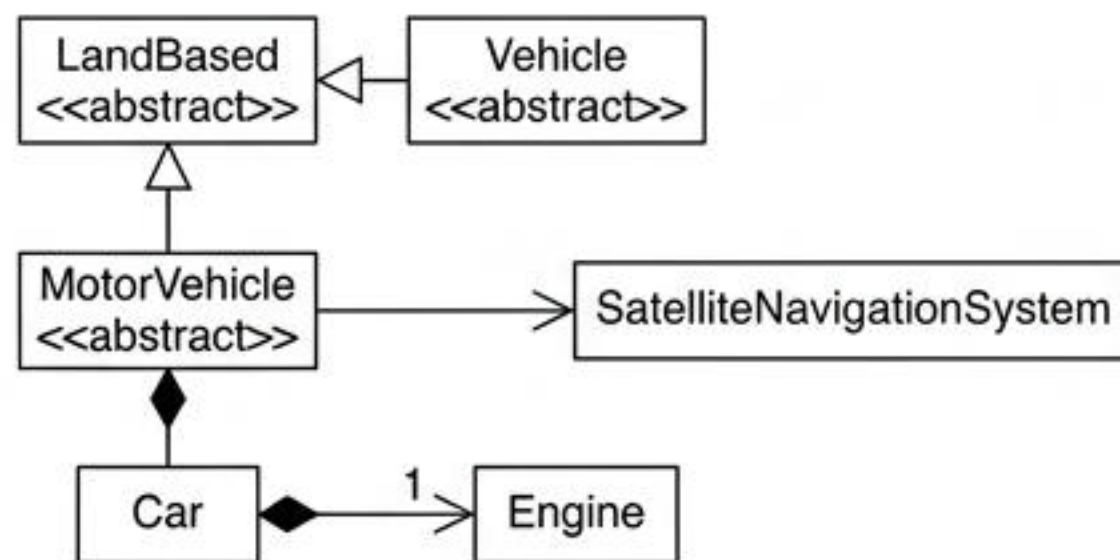
El Plano Arquitectónico para la Flexibilidad y Escalabilidad del Software



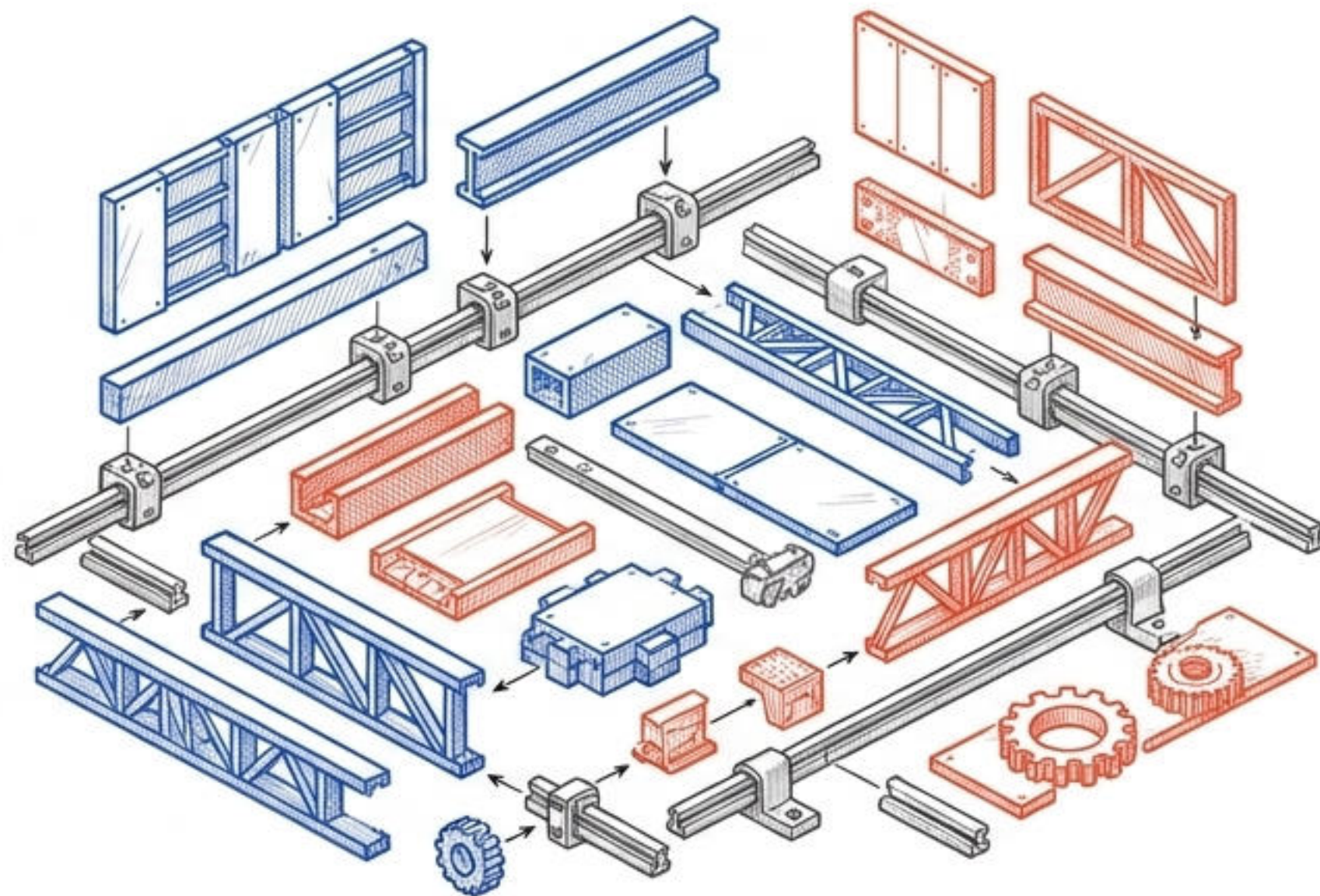
La Trampa de la Herencia



- **Dependencia Fuerte:** Crea una relación estricta "Es-un".
- **Deuda Técnica:** Un cambio en la clase base afecta y puede quebrar toda la jerarquía.
- **Acoplamiento Alto:** Difícil de modificar en sistemas complejos.



La Solución Modular



- **Estandarización:** Uso de Interfaces para definir contratos.
- **Composición:** Uso de Delegación para intercambiar roles dinámicamente.
- **Acoplamiento Bajo:** Piezas independientes y mantenibles.

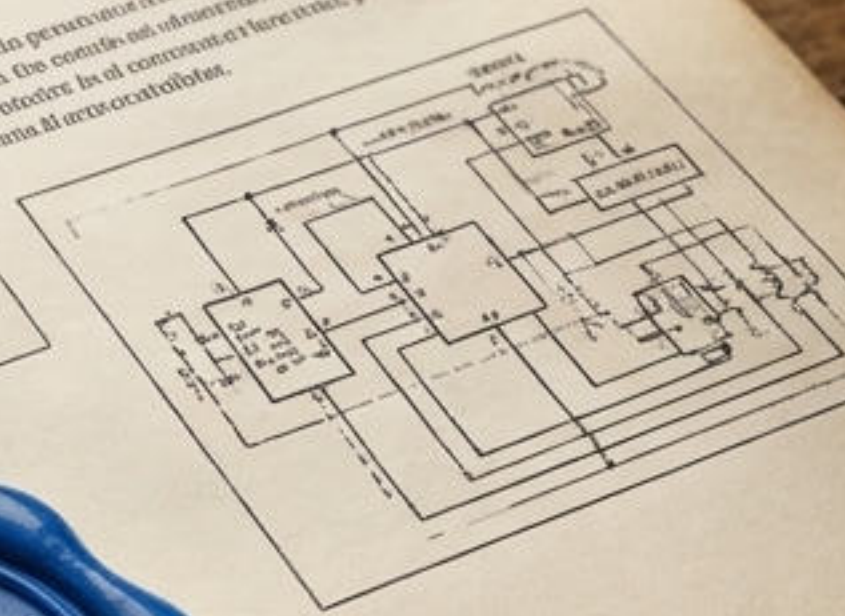
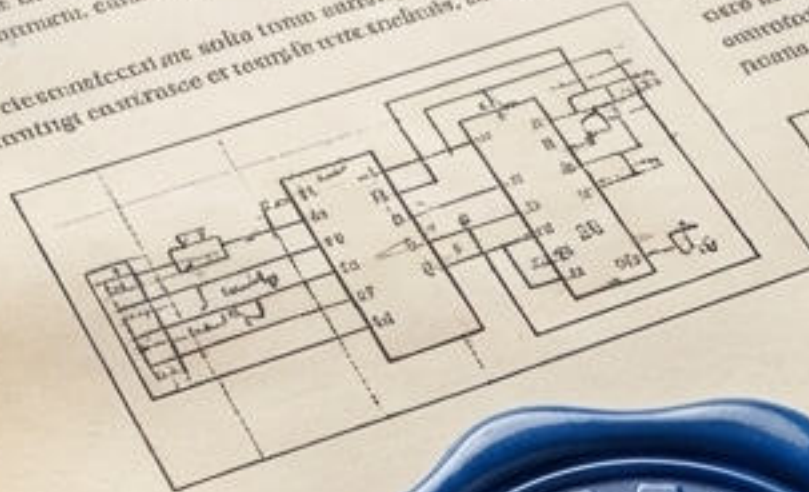
Legal Contract

The legal is a contract between two parties, one of which obligates the other to perform a certain action or abstain from a certain action, and is often a binding agreement between two parties.

On the other hand, a contract is a legal agreement between two or more parties that is enforceable by law. It is a promise or set of promises for which the law provides a remedy.

So, in essence, a contract is a legal agreement between two or more parties that is enforceable by law. It is a promise or set of promises for which the law provides a remedy.

This is a legal contract between two parties, one of which obligates the other to perform a certain action or abstain from a certain action, and is often a binding agreement between two parties.



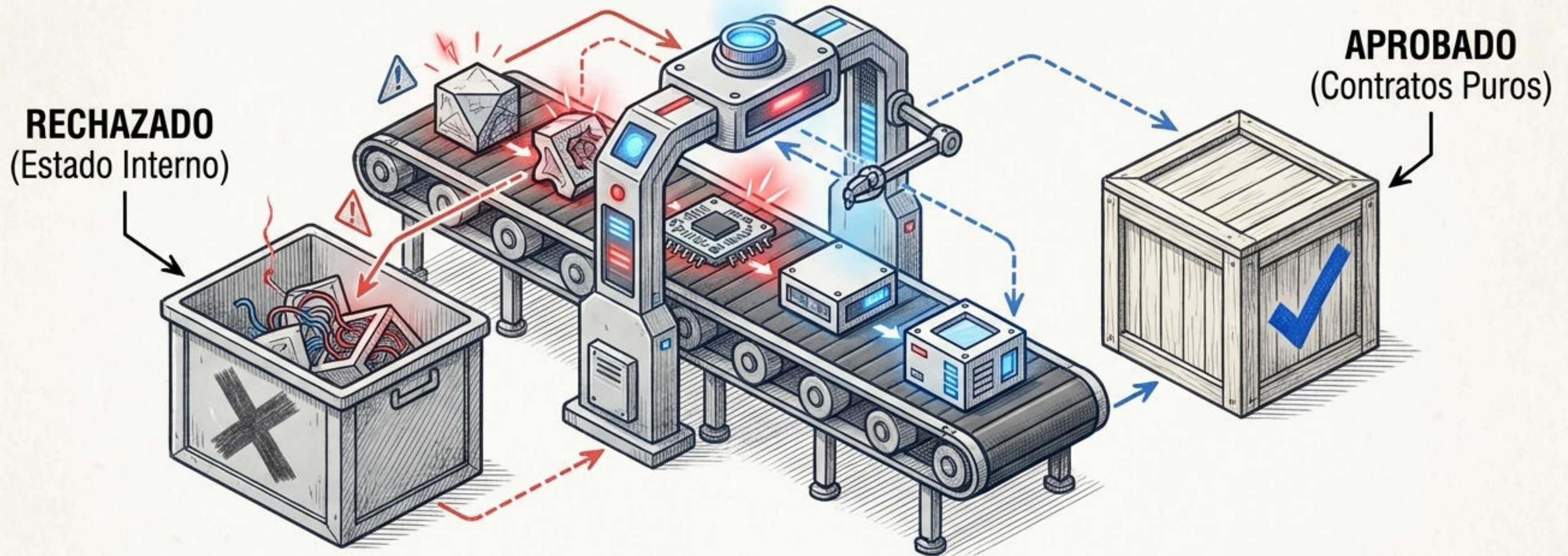
La Interfaz: El Contrato Comportamental

Definición: Una clase abstracta pura que actúa como un **contrato irrompible** entre el productor y el consumidor.

Principio: Define estrictamente el **QUÉ** debe hacer un objeto, pero delega completamente el **CÓMO** a la clase que lo implementa.

Ventaja Gerencial: Permite el trabajo en paralelo. El Equipo A programa contra la Interfaz; el Equipo B desarrolla la implementación oculta.

Anatomía de la Interfaz: Reglas del Compilador



- Campos variables (variables de instancia)
- Métodos con lógica implementada
- **Razón:** La interfaz prohíbe mantener un estado.

- Constantes estáticas (static final)
- Firmas de métodos 100% abstractos
- **Razón:** Solo se garantiza el comportamiento.

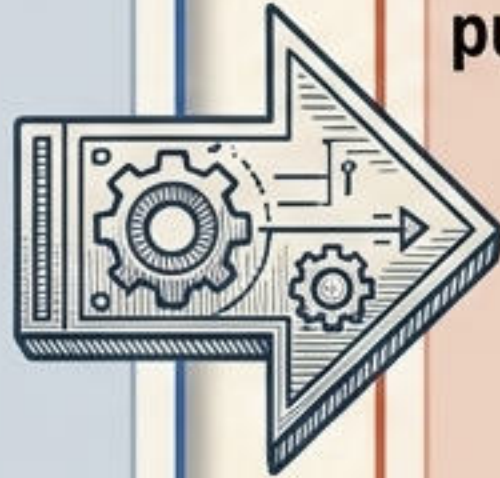
Tutorial: Implementando la Interfaz

Paso 1: El Productor (Contrato)

```
// Archivo: Volador.java
public interface Volador {

    // 100% abstracto: sin cuerpo
    void despegar();

}
```



Paso 2: El Consumidor (Implementación)

```
// Archivo: Dron.java
public class Dron implements Volador {

    // Implementación obligatoria
    public void despegar() {
        encenderRotores();
    }

}
```

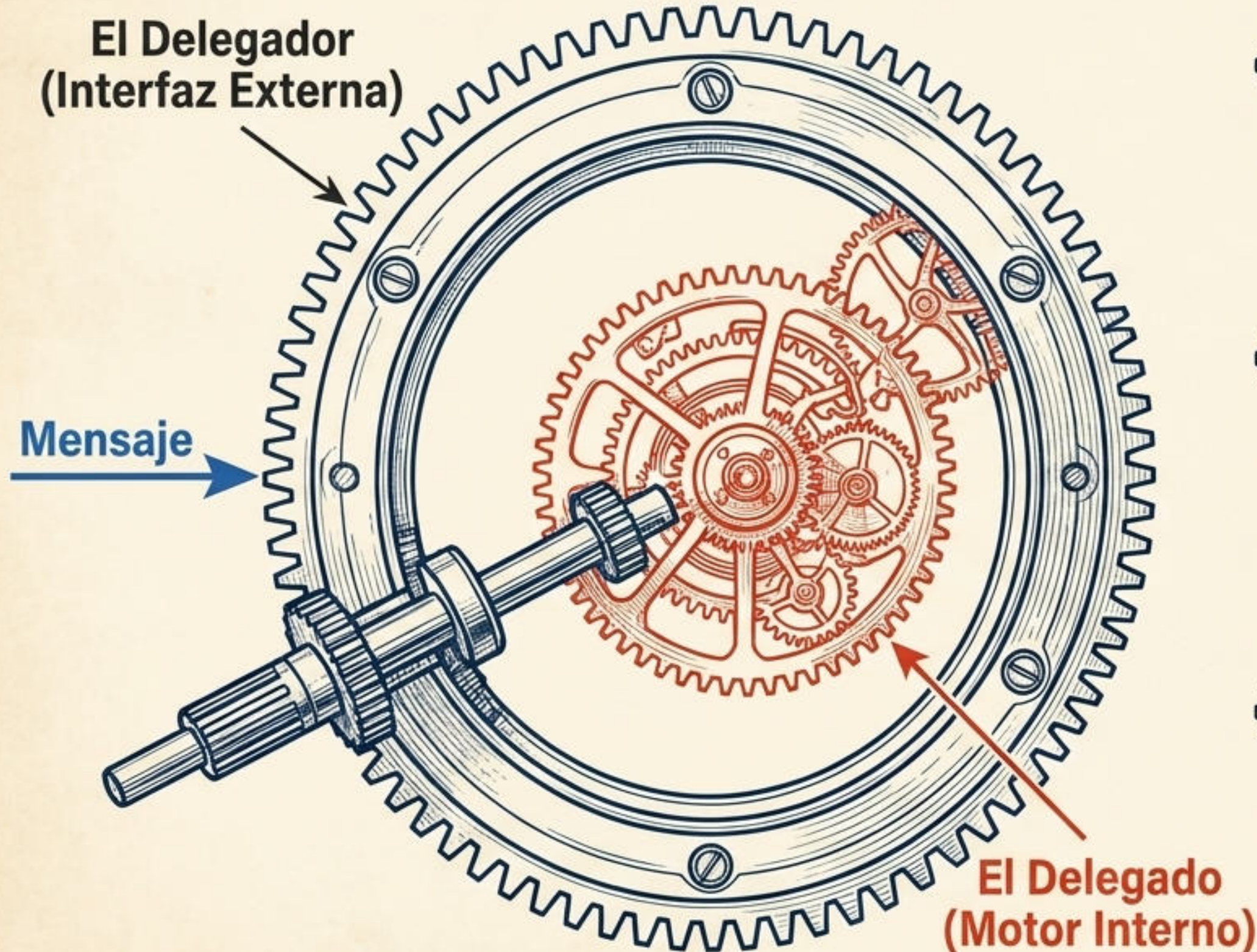
Polimorfismo Transversal sin Herencia

El Poder del Enlace Tardío (Late Binding)



Clases sin relación lógica pueden agruparse y recibir el mismo mensaje "despegar()".
El sistema conecta la versión correcta en tiempo de ejecución.

Delegación: El Patrón del Emisario



- ⚙️ **Concepto:** Transferir la responsabilidad de una tarea a un objeto auxiliar oculto internamente.
- ⚙️ **Filosofía:** En lugar de "Yo SOY un especialista" (Herencia), la delegación afirma "Yo TENGO un especialista y le paso el trabajo" (Composición).
- ⚙️ **Beneficio:** Flexibilidad extrema. El delegado puede ser intercambiado dinámicamente sin alterar la fachada exterior.

Tutorial: Configurando un Delegado

Paso 1: El Especialista (Delegado)

```
// Paso 1: El Especialista (Delegado)
public class MotorCalculo {
    public void procesar() {
        /* lógica algorítmica compleja */
    }
}
```

Paso 2: El Emisario (Delegador)

```
// Paso 2: El Emisario (Delegador)
public class InterfazUsuario {
    // Relación Tiene-un (Composición)
    private MotorCalculo miDelegado = new MotorCalculo();

    public void ejecutarTarea() {
        // Redirección del mensaje
        miDelegado.procesar();
    }
}
```

El objeto delegador contiene y gestiona su propia instancia del trabajador.

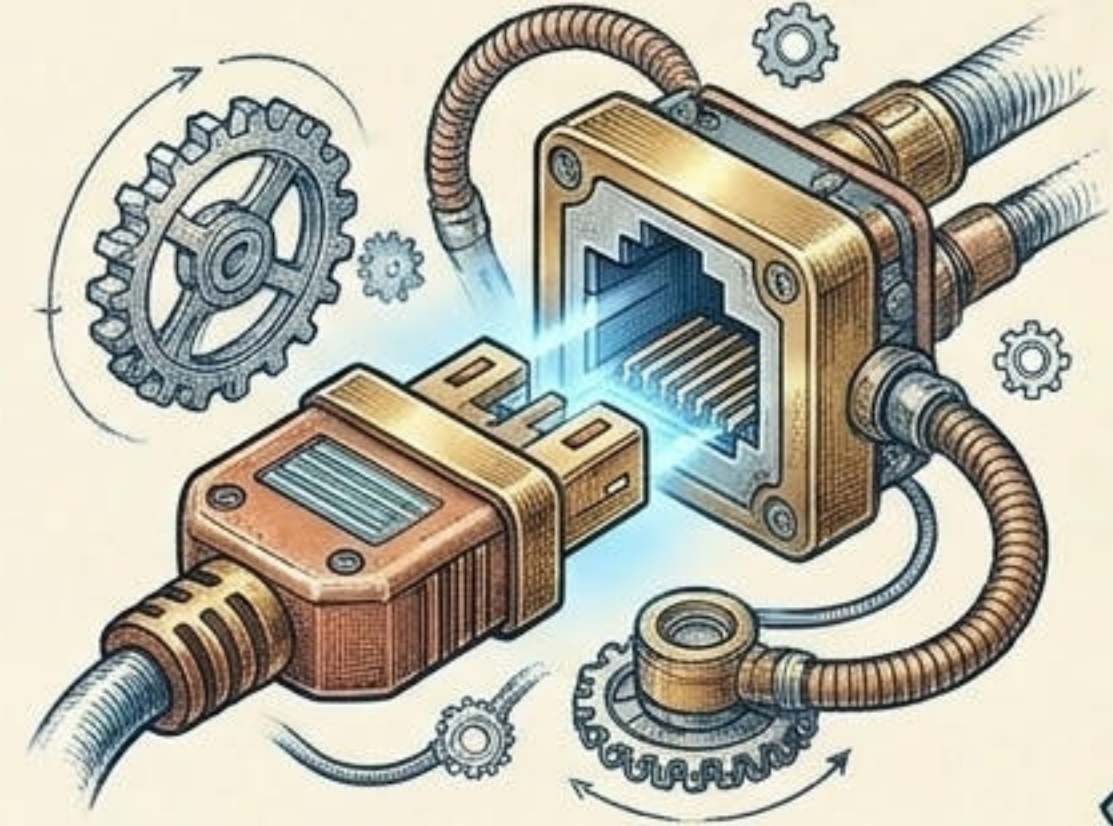
La Decisión: Herencia vs. Delegación

Herencia



- ⚙️ **Relación:** Es-Un (Is-A)
- ⚙️ **Acoplamiento:** Fuerte y rígido
- ⚙️ **Tiempo de decisión:** Compilación (Estático)
- ⚙️ **Regla Arquitectónica:** Solo usar cuando un objeto es verdaderamente una extensión biológica o lógica indiscutible del otro.

Delegación



- ⚙️ **Relación:** Tiene-Un / Usa-Un
- ⚙️ **Acoplamiento:** Débil y modular
- ⚙️ **Tiempo de decisión:** Ejecución (Dinámico)
- ⚙️ **Regla Arquitectónica:** Usar por defecto para ensamblar funcionalidades complejas, compartir comportamiento y preservar el encapsulamiento.

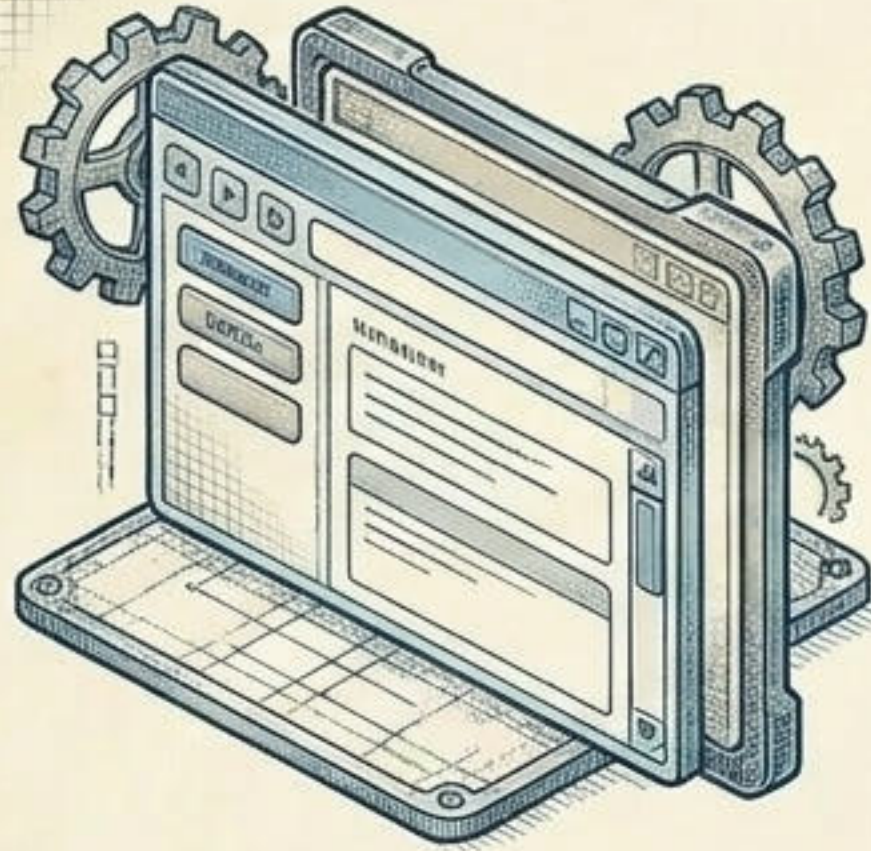
Matriz de Decisión Arquitectónica

Herramienta	Tipo de Relación	¿Comparte Estado?	¿Comparte Código?	Riesgo de Acoplamiento
Herencia	Es-Un	✓ Sí	✓ Sí	⚠ Alto (Peligro)
Interfaz	Actúa-Como	✗ No (Solo constantes)	✗ No (Contrato vacío)	🛡 Bajo (Óptimo)
Delegación	Tiene-Un	✓ Sí (Vía el delegado)	✓ Sí (Vía redirección)	📊 Medio (Modular)

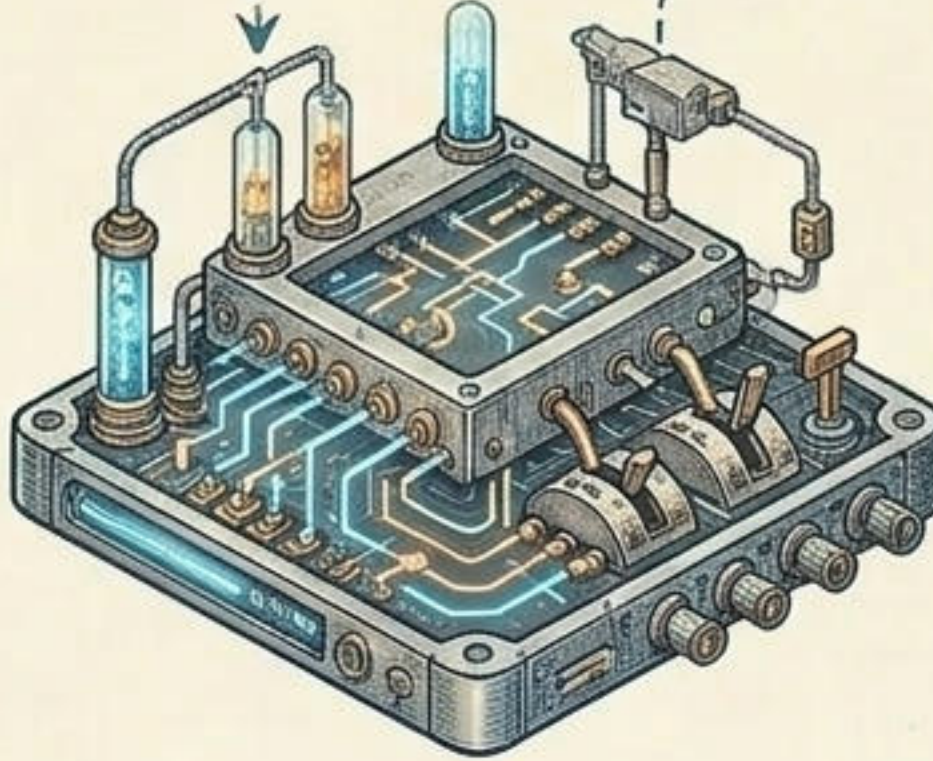
Síntesis: Los arquitectos de software modernos prefieren Interfaces y Delegación para minimizar la deuda técnica y el riesgo de refactorización masiva.

Aplicación Real: El Patrón MVC

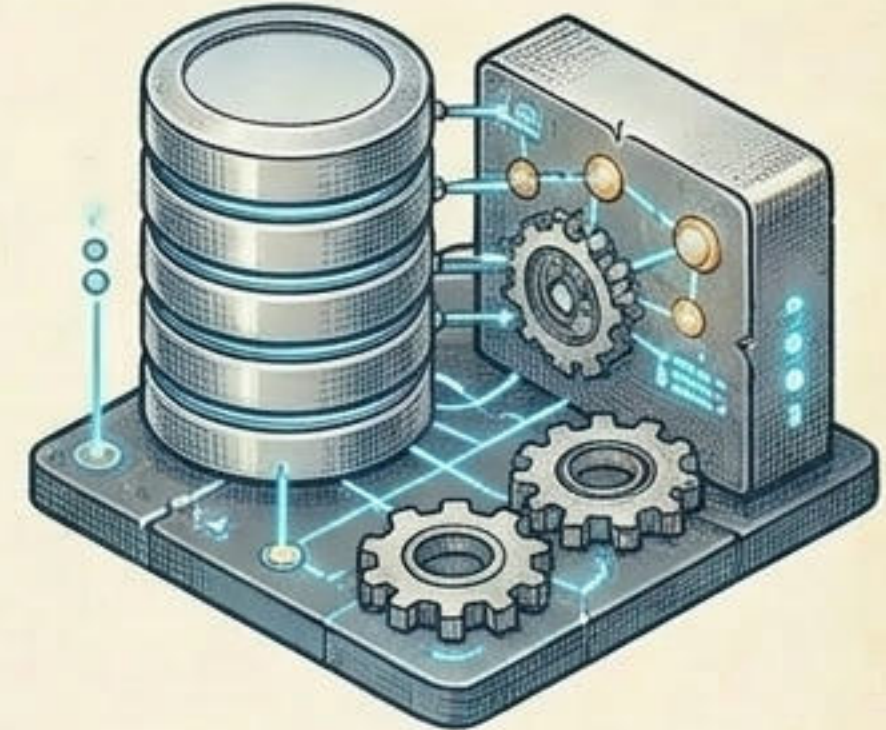
Separación de roles en Interfaces Gráficas (GUIs)



VISTA: Presentación gráfica.
Solo muestra datos.



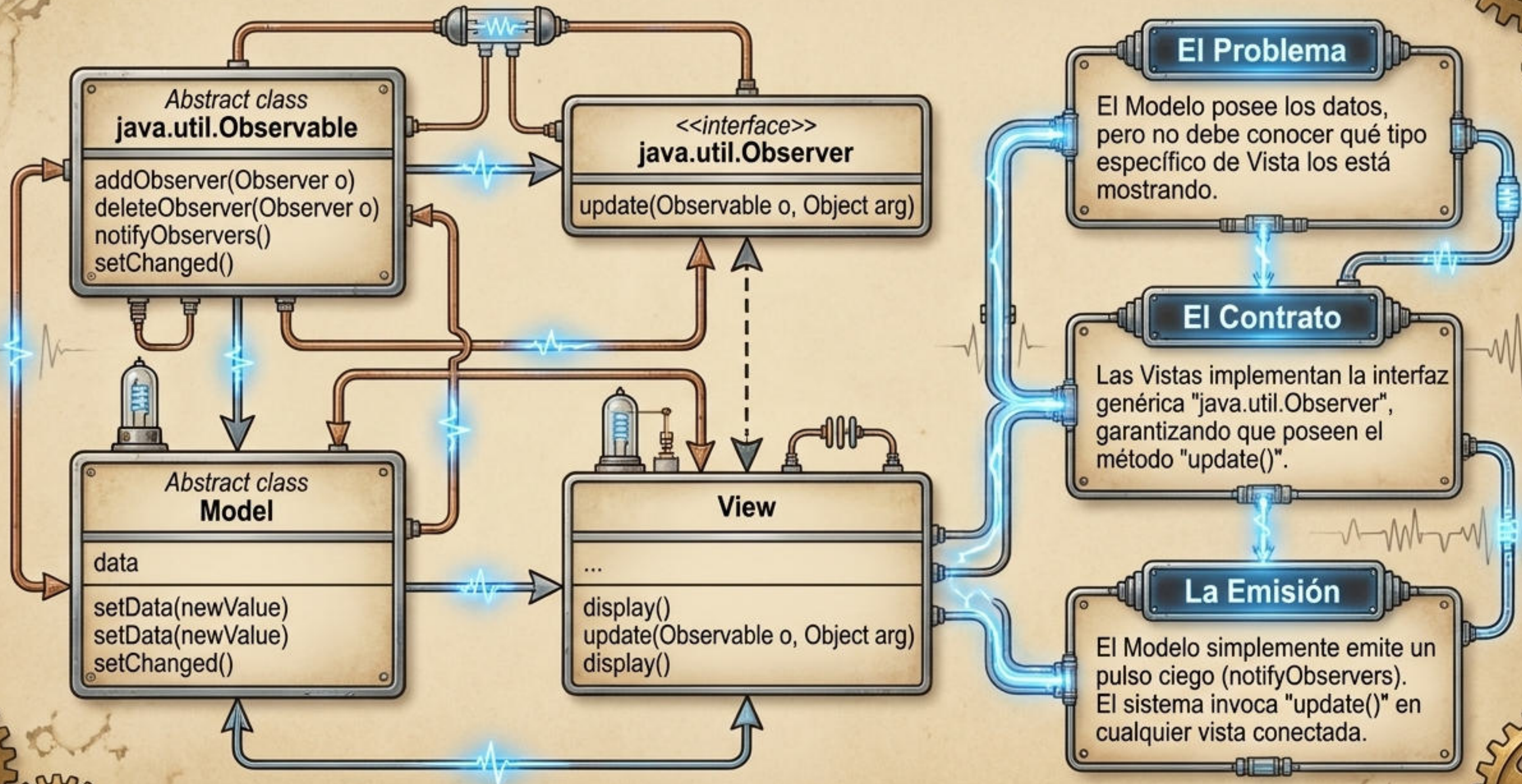
CONTROLADOR: Intermediario.
Maneja eventos del mouse y teclado.



MODELO: Reglas de negocio
y estado interno puro.

El Reto: ¿Cómo comunicamos estas tres capas
sin crear código espagueti inmanejable?
La respuesta reside en Interfaces y Delegación.

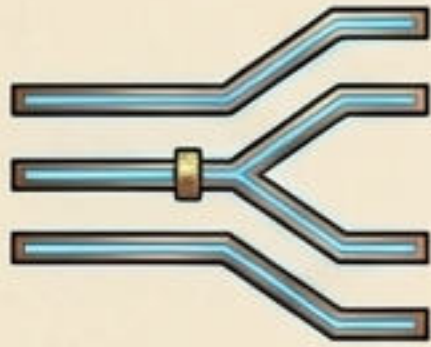
MVC y las Interfaces: El Enfoque Observer



MVC y la Delegación: Java Beans

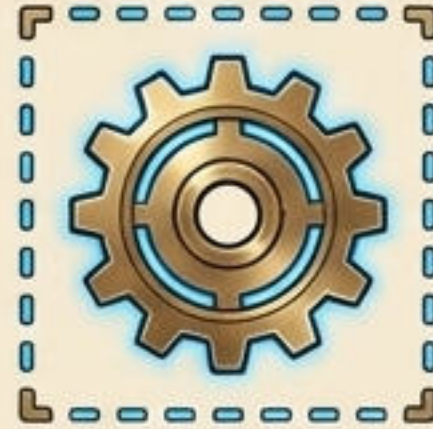


La Visión Gerencial: El ROI de la Arquitectura



Paralelización del Trabajo

Las Interfaces actúan como contratos irrompibles de API. El equipo de Backend y el de Frontend pueden construir simultáneamente sin esperarse el uno al otro.



Mantenimiento a Bajo Costo

La Delegación aísla los componentes. Actualizar o reemplazar un motor algorítmico interno no requiere reescribir toda la aplicación.



Reducción de Deuda Técnica

Evitar la herencia profunda significa que actualizaciones futuras en lenguajes o frameworks no colapsarán la base central de su producto.

Las Piedras Angulares de la Flexibilidad

LA INTERFAZ

- Diseñe por contrato.
- Exija estandarización sin jerarquías.

LA DELEGACIÓN

- Transfiera responsabilidades.
- Ensable sistemas dinámicos.

El Resultado: Software estructurado para funcionar hoy y escalar elegantemente hacia el futuro.